

ADOPT-04: Team Enablement

Series: ADOPT — Observability Adoption & Maturity | **Notebook:** 4 of 6 | **Created:** March 2026 | **Last Updated:** 07/24/2026

Overview

Technology alone does not drive observability maturity — people do. This notebook provides a framework for team enablement: role-based learning paths, recommended notebook study sequences, skill assessment approaches, and strategies for building internal champions. The goal is to move from a small group of Dynatrace experts to broad organizational capability.

Table of Contents

1. [The Enablement Challenge](#)
 2. [Role-Based Learning Paths](#)
 3. [Recommended Study Sequences](#)
 4. [Skill Assessment Framework](#)
 5. [Building Internal Champions](#)
 6. [Practical Exercises — Exploring Your Environment](#)
 7. [Training Roadmap Template](#)
 8. [Summary and Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	<code>storage:logs:read</code> , <code>storage:metrics:read</code> , <code>storage:entities:read</code>
Audience	Engineering managers, platform team leads, learning & development
Context	Completed ADOPT-01 (maturity model) and ADOPT-02 (platform health) assessments

1. The Enablement Challenge

A common failure mode in observability adoption is the **"hero problem"** — one or two people become the Dynatrace experts, and everyone else depends on them. This creates bottlenecks, single points of failure, and prevents the organization from scaling its observability practice.

Signs of an Enablement Gap

Symptom	Root Cause
Same 2-3 people answer all Dynatrace questions	Knowledge is concentrated, not distributed
Teams open tickets instead of investigating themselves	Lack of DQL skills and self-service capability
Dashboards are created by one team, consumed by others	Dashboard consumers cannot create their own views
New features are not adopted after rollout	No structured learning path for new capabilities
Incidents still require manual log SSH	Teams do not know how to use Grail for investigation

The Goal

Move from a model where knowledge is concentrated to one where every team has at least one member who can:

- Write basic DQL queries
- Navigate the Dynatrace UI independently
- Create dashboards for their service
- Respond to Dynatrace Intelligence problems without escalation

2. Role-Based Learning Paths

Different roles need different depths of Dynatrace knowledge. The following learning paths define what each role should be able to do.

2.1 SRE / Operations Engineer

Skill Level	Capabilities
Beginner	Navigate the Dynatrace UI, understand detected problems, view dashboards
Intermediate	Write DQL queries, investigate incidents with logs/traces, create alerting rules
Advanced	Build custom dashboards, configure SLOs, optimize OpenPipeline routing
Expert	Design automation workflows, implement self-healing, manage IAM policies

2.2 Application Developer

Skill Level	Capabilities
Beginner	View service health, understand distributed traces, read span data
Intermediate	Query spans and logs for their services, understand error rates
Advanced	Instrument custom spans/metrics, configure code-level monitoring
Expert	Design observability-first applications, implement OpenTelemetry

2.3 Platform Engineer

Skill Level	Capabilities
Beginner	Understand Dynatrace architecture, deploy OneAgent, configure basic settings
Intermediate	Manage Grail buckets, configure OpenPipeline, set up IAM
Advanced	Implement configuration-as-code (Monaco), design multi-tenant IAM
Expert	Build custom Dynatrace apps, integrate with CI/CD, manage at scale

2.4 Engineering Manager

Skill Level	Capabilities
Beginner	Read dashboards, understand SLO status, interpret problem impact
Intermediate	Define SLOs for their team's services, review MTTR/MTTD trends
Advanced	Drive observability culture, allocate resources for monitoring investment

3. Recommended Study Sequences

Map roles to notebook series from this repository for structured learning.

3.1 SRE / Operations Track

Order	Series	Focus	Duration
1	ONBRD	Dynatrace fundamentals	2 weeks
2	OPLOGS	Log management with OpenPipeline	1 week
3	SPANS	Distributed tracing	1 week
4	WFLOW	Workflows and alert notifications	1 week
5	SYNTH	Synthetic monitoring	1 week

3.2 Developer Track

Order	Series	Focus	Duration
1	ONBRD (notebooks 1-5)	Dynatrace basics	1 week
2	SPANS	Understanding distributed traces	1 week
3	OTEL	OpenTelemetry integration	1 week
4	ORGNZ (notebooks 1-3)	Data organization basics	3 days

3.3 Platform Engineer Track

Order	Series	Focus	Duration
1	ONBRD	Full onboarding series	2 weeks
2	K8S	Kubernetes monitoring	2 weeks
3	IAM	Identity and access management	1 week
4	AUTOM	Configuration automation	1 week
5	ORGNZ	Full data organization series	2 weeks
6	OPMIG	OpenPipeline migration	1 week

3.4 Manager Track

Order	Series	Focus	Duration
1	ADOPT (this series)	Strategy and metrics	1 week
2	ONBRD (notebooks 1-3)	Platform overview	3 days
3	ORGNZ (notebook 1)	Data organization concepts	1 day

4. Skill Assessment Framework

Use practical DQL exercises to assess current skill levels. The following categories test progressively deeper understanding.

Level 1: Navigation (Beginner)

- Can the person find and explain a detected problem?
- Can they navigate to a specific host or service in the UI?
- Can they read a dashboard and explain what each tile shows?

Level 2: Basic DQL (Intermediate)

- Can they write a `fetch logs` query with time range and filters?
- Can they count records and group by a field?
- Can they create a basic timeseries chart?

Level 3: Applied DQL (Advanced)

- Can they join data from two sources (e.g., logs + entities)?
- Can they calculate MTTR from detected problems?
- Can they build a multi-tile dashboard from DQL queries?

Level 4: Platform Operations (Expert)

- Can they configure OpenPipeline routing rules?
- Can they design a Grail bucket strategy?
- Can they create a Workflow for automated remediation?

5. Building Internal Champions

Internal champions are the force multiplier for observability adoption. They are team members who develop deep Dynatrace expertise and actively help others.

Champion Program Structure

Phase	Activities	Duration
Selection	Identify 1-2 motivated individuals per team	Week 1
Deep Training	Complete platform engineer track + hands-on labs	Weeks 2-6

Phase	Activities	Duration
Shadowing	Pair with existing experts during incident response	Weeks 7-8
Teaching	Champion leads a knowledge-sharing session for their team	Week 9
Ongoing	Monthly champion sync, shared Slack channel, quarterly skills refresh	Continuous

Champion Responsibilities

- First point of contact for Dynatrace questions within their team
- Create and maintain team-specific dashboards
- Review and improve alerting configurations quarterly
- Attend monthly platform team syncs
- Share learnings back to the broader champion community

Measuring Champion Effectiveness

Metric	Target
Escalation tickets to platform team	Decrease by 50% within 3 months
Teams with self-service dashboards	100% within 6 months
DQL-literate team members	At least 2 per team within 6 months

6. Practical Exercises — Exploring Your Environment

The following DQL queries serve as starter exercises for new users. Each one answers a practical question about the environment.

Exercise 1: "What services are running in my environment?"

```
// List all monitored services with their types
fetch dt.entity.service
| fieldsAdd entity.name, serviceType
| sort entity.name asc
| limit 25

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | fieldsAdd name, dt.service.sdv1_type
//   | sort name asc
//   | limit 25
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities than
// the classic entity store; for a pre-migration discovery inventory keep the
// classic query above.
// Field maps: serviceType -> dt.service.sdv1_type; entity.name -> name.
```

Exercise 2: "What happened in the last hour?"

```
// Count error logs in the last hour by log level
fetch logs, from:-1h
| summarize log_count = count(), by:{loglevel}
| sort log_count desc
```

Exercise 3: "Which hosts are using the most CPU?"

```
// Top 5 hosts by CPU usage in the last hour
timeseries cpuUsage = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpu = arrayAvg(cpuUsage)
| sort avgCpu desc
| limit 5
```


Exercise 4: "Are there any active problems?"

```
// List currently active detected problems
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| fieldsKeep display_id, event.name, event.category, event.status, event.start
| sort event.start desc
```

Exercise 5: "How much log data are we ingesting?"

```
// Log ingestion volume over the last 24 hours by source
fetch logs, from:-24h
| summarize record_count = count(), by:{log.source}
| sort record_count desc
| limit 10
```

7. Training Roadmap Template

Use this template to plan a 12-week enablement program for your organization.

Week	Focus Area	Activities	Deliverable
1-2	Platform Orientation	UI walkthrough, basic navigation, Dynatrace Intelligence overview	All team leads can navigate Dynatrace
3-4	DQL Fundamentals	Exercises 1-5 above, basic fetch/filter/summarize	All participants can write a basic query
5-6	Role-Specific Deep Dives	SRE: logs + alerting. Dev: traces + spans. Platform: IAM + config	Role-specific skills demonstrated
7-8	Dashboard Building	Create team dashboards from DQL queries	Each team has a service dashboard
9-10	Incident Response Practice	Simulated incident using real detected problems	Teams can triage without escalation
11-12	Champion Certification	Champions lead a knowledge-sharing session	Champion program officially launched

Success Criteria

Metric	12-Week Target
Teams with at least 1 DQL-literate member	100%
Teams with self-service dashboards	> 80%
Escalation tickets to platform team	30% reduction
Champions identified and trained	1 per team

8. Summary and Next Steps

Key Takeaways

- Observability adoption fails without team enablement — technology alone is not enough
- Role-based learning paths ensure each person learns what they need, not everything
- Internal champions are the most effective way to scale observability knowledge
- Practical DQL exercises build confidence faster than documentation alone
- A structured 12-week program can transform an organization's observability capability

Next Steps

- Proceed to **ADOPT-05: Optimization Roadmap** to plan your long-term observability investment
- Identify champion candidates within each team
- Schedule the first "Platform Orientation" session for weeks 1-2
- Create a shared Slack/Teams channel for Dynatrace knowledge sharing

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.